

```
# python torch1.py
['be', 'the', 'change', 'you', 'wish', 'to', 'see', 'in', 'the', 'world']
['be', 'the', 'change', 'you', 'wish', 'to', 'see', 'in', 'the', 'world', 'be
the', 'the change', 'change you', 'you wish', 'wish to', 'to see', 'see in',
'in the', 'the world']
```

Figure 3: PyTorch for NLP

that uses specialised memory cells and gates to selectively remember or forget information over time. Thus, RNN and LSTM can be referred to as feedback neural networks. Both are neural networks specifically designed to process sequential data such as natural language text. LSTMs can be implemented in Python using TensorFlow/Keras or PyTorch. The class `tf.keras.layers.LSTM()` generates LSTM layers using TensorFlow/Keras and the class `torch.nn.LSTM()` generates LSTM layers using PyTorch. Notice that FNNs, CNNs, RNNs, and LSTMs are used to implement supervised learning models.

Let us now turn our attention to generative adversarial networks (GAN), which are a specific type of neural network used for generative modelling. A GAN is composed of two separate neural networks -- a generator and a discriminator. The generator is trained to create new data that resembles the training data, while the discriminator is trained to distinguish between the generated data and real data. It is worth noting that GANs are distinct from the other neural networks we have discussed so far in that they are used for unsupervised learning. GANs can be implemented in Python using TensorFlow/Keras or PyTorch. GANs typically rely on a variety of different types of layers, including dense layers, convolutional layers, convolutional transpose layers, activation layers, and dropout layers. For example, in TensorFlow/Keras, the `tf.keras.layers` module provides a range of pre-built layers that can be used for implementing GANs, while PyTorch offers similar functionality through its `torch.nn` module.

Let us now turn our attention to

autoencoders. They are another type of neural network used for unsupervised learning. An autoencoder is made up of two parts, an encoder and a decoder. Autoencoders are trained to reconstruct the input data. They are often used for tasks such as anomaly detection and dimensionality reduction. Python offers several ways to implement autoencoders, which includes usage of TensorFlow/Keras or PyTorch. The implementation of autoencoders can be done by using the `tf.keras.module` module in TensorFlow/Keras or by using the `torch.nn` module in PyTorch.

Besides the neural networks that we have discussed, there are numerous other notable neural networks to be considered. Some of these include the Hopfield network, which was introduced in 1982, and is a type of recurrent neural network that stores and retrieves patterns; Boltzmann machines (BM), which are stochastic neural networks that represent complex probability distributions; restricted Boltzmann machines (RBM) introduced in 1986, which are Boltzmann machines with a restricted architecture that facilitates efficient training; and deep belief networks (DBNs), which are generative models with multiple layers of RBMs. Let us now shift our focus to the more practical aspects of developing applications based on AI and machine learning.

NLP using PyTorch

First, let us discuss natural language processing or NLP, which plays a critical role in automation. Notice that NLP faces significant challenges due to the sheer number of languages spoken globally — eight languages are spoken

by more than 100 million people each, and 27 languages are spoken by at least 50 million people each (source: Wikipedia). These languages differ in their grammatical structures, idiomatic expressions, and linguistic features, making analysis and modelling difficult. Additionally, in our globalised world, text often contains words from multiple languages, some of which may even be from relatively obscure languages. Processing the vast amount of data involved in NLP presents another significant challenge, along with the inherent ambiguity of natural languages. Despite these difficulties, we have access to excellent tools for NLP such as Python powered with PyTorch. Now, let us briefly discuss the use of PyTorch for NLP.

Recall that we have already seen examples of how PyTorch can be used for building neural networks, which are commonly employed in NLP. In this section, we will examine some of the key PyTorch modules and classes that are particularly useful for NLP. Specifically, the `torch.nn` module offers a range of classes for building neural networks such as CNN, RNN, etc, as well as activation functions like ReLU, Sigmoid, etc. The `torch.optim` module provides various optimisation algorithms such as stochastic gradient descent (SGD), RMSprop, Adam, etc, which can be used to train neural networks. Recall that we used the Adam optimisation algorithm in our neural network model to classify images of handwritten digits. However, keep in mind that our model was implemented using Keras. The `torch.utils.data` module provides a set of tools for working